

# Docker

---

## ■ Dockerfile

- Un Dockerfile è un documento di testo che contiene tutti i comandi che un utente potrebbe eseguire da riga di comando per assemblare un'immagine
- Contiene una sequenza di istruzioni eseguite in ordine
- Permette di automatizzare e standardizzare la creazione dell'ambiente
- Supporto la portabilità tra sistemi

## ■ Istruzioni principali

- **FROM** – immagine di base
- **COPY / ADD** – copia dei file
- **RUN** – esecuzione di comandi durante la build
- **EXPOSE** – porte esposte
- **CMD / ENTRYPOINT** – comando di avvio del container

# Docker

---

- **Esempio precedente ...**
  - server-bin.c e client-bin.c

- **Dockerfile server:**

```
FROM ubuntu:22.04
```

```
# Compilatore per costruire il binario  
RUN apt update && apt install -y gcc
```

```
WORKDIR /app
```

```
# Copio il sorgente C nell'immagine  
COPY server-bin.c .
```

```
# Compilo dentro l'immagine  
RUN gcc server-bin.c -o server-bin
```

```
# Quando il container parte, esegue direttamente il server  
CMD ["/server-bin"]
```

# Docker

---

- **Esempio precedente ...**
  - server-bin.c e client-bin.c

- **Dockerfile client:**

```
FROM ubuntu:22.04
```

```
RUN apt update && apt install -y gcc
```

```
WORKDIR /app
```

```
COPY client-bin.c .
```

```
RUN gcc client-bin.c -o client-bin
```

```
# Default: si connette a "server-cont"  
CMD ["/client-bin", "server-cont"]
```

# Docker

## ■ Esempio precedente ...

- server-bin.c e client-bin.c

## ■ Dockerfile client:

```
FROM ubuntu:22.04
```

```
RUN apt update && apt install -y gcc
```

```
WORKDIR /app
```

```
COPY client-bin.c .
```

```
RUN gcc client-bin.c -o client-bin
```

```
# Default: si connette a "server-cont"  
CMD ["/client-bin", "server-cont"]
```

COPY copia file SOLO dal build context (la directory del docker build)

Con docker build, Docker costruisce una nuova immagine a layer:

layer base (FROM ubuntu)



layer RUN (es: apt install)



layer COPY (client-bin.c o server-bin.c)



layer RUN (compilazione)

Quando parte il container:

- prende tutti i layer immagine
- li monta in sola lettura
- aggiunge writable layer del container

COPY in Dockerfile diverso da

`docker cp file container:/path`

Che agisce nel container

RUN e COPY creano layer IMMUTABILI (build-time)

CMD non crea layer (runtime)

# Docker

---

## ■ Caching layer e invalidazione

- Quando Docker esegue un build, ogni istruzione del Dockerfile produce un layer immutabile

## ■ Cosa viene messo in cache?

- FROM ubuntu → layer base
- RUN apt install → layer con tutti i pacchetti già installati
- COPY client-bin.c . → layer con il file
- RUN gcc ... → layer con il binario compilato

## ■ Se una istruzione NON cambia, Docker riutilizza il layer precedente

- Docker NON ricostruisce TUTTA l'immagine, SOLO ciò che viene dopo il primo layer differente.

# Docker

---

- **Esempio precedente ...**

- server-bin.c e client-bin.c

Il file va in una **cartella dedicata**, insieme ai file da copiare nell'immagine

- **Server:**

```
es-client-server/  
└── server/  
    ├── server-bin.c  
    └── Dockerfile
```

- **Client:**

```
es-client-server/  
└── client/  
    ├── client-bin.c  
    └── Dockerfile
```

- **Immagine:**

```
cd es-client-server/server  
docker build -t c-server .
```

```
cd es-client-server/client  
docker build -t c-client .
```

# Docker

## ■ Esempio precedente ...

- server-bin.c e client-bin.c

## ■ Server:

```
es-client-server/  
└── server/  
    ├── server-bin.c  
    └── Dockerfile
```

## ■ Client:

```
es-client-server/  
└── client/  
    ├── client-bin.c  
    └── Dockerfile
```

## ■ Immagine:

```
cd es-client-server/server  
docker build -t c-server .
```

```
cd es-client-server/client  
docker build -t c-client .
```

Il file va in una **cartella dedicata**, insieme ai file da copiare nell'immagine

Non richiesto da docker, che supporta anche:

```
docker build -f Dockerfile.server .  
docker build -f Dockerfile.client .
```

Ma è buona pratica:

- ogni servizio ha il suo build context
- i file restano isolati
- la build cache è più efficiente
- i Dockerfile sono auto-contenuti
- compose diventa più leggibile

# Docker

---

- **Dockerfile**

- **Pipeline**

Dockerfile → docker build → immagine → docker run → container → processi

- **Immagine**

Il Dockerfile è il "sorgente" di un'immagine Docker

**docker build -t nome .**

trasforma il Dockerfile in un'immagine con quel nome

- **Container**

lanciando

**docker run nome**

si crea un container che esegue quella immagine

# Docker

---

- **Dockerfile**

- **Pipeline**

Dockerfile → docker build → immagine → docker run → container → processi

- **Immagine**

```
cd es-client-server/server  
docker build -t c-server .
```

```
cd es-client-server/client  
docker build -t c-client .
```

- **Rete**

```
docker network create miarete
```

- **Container**

```
docker run --rm --name server-cont --network miarete c-server  
docker run --rm --name client-cont --network miarete c-client server-cont
```

# Docker

---

- **Dockerfile**
- **Recuperare il PID reale del container**

```
docker inspect server-cont --format '{{.State.Pid}}'
```

- **Output**

```
2347
```

- **Esplorare in namespace reali**

```
ls -l /proc/2347/ns
```

- Ma WSL2 non li mostra ...
  - Il kernel che gestisce è **docker-desktop**

# Docker

---

## ■ Docker-compose

- Permette di descrivere più container in un unico file (docker-compose.yml)
- Definire la reti, immagini, volumi e comandi
- Avviare tutto insieme

## ■ Esempio

```
services:
  server:
    build: ./server
    container_name: server-cont
    networks:
      - rete-mia

  client:
    build: ./client
    container_name: client-cont
    networks:
      - miarete
    depends_on:
      - server    # il client parte dopo il server
    command: ["server-cont"]
networks:
  rete-mia:
```

# Docker

---

## ■ Docker-compose

- Permette di descrivere più container in un unico file (docker-compose.yml)
- Definire la reti, immagini, volumi e comandi
- Avviare tutto insieme

## ■ Services:

- modello astratto di container
- permette di costruire container dello stesso tipo
- in un service si può definire:

- immagine da usare (image:) o da costruire (build:)
- l'entrypoint o i comandi (command:)
- le variabili (environment:)
- le reti (networks:)
- i volumi (volumes:)
- le dipendenze (depends\_on:)

```
services:  
  client:  
    build: ./client  
    command: ["server"]
```

# Docker

---

## ■ Docker-compose

- Permette di descrivere più container in un unico file (docker-compose.yml)
- Definire la reti, immagini, volumi e comandi
- Avviare tutto insieme

## ■ Avvio servizi

- `docker compose build <servizio>`
- `docker compose up`
  - Build immagini
  - Creazione rete
  - Avvio container
  - Logging combinato
- `docker compose down`
  - chiude

```
services:
  server:
    build: ./server
    container_name: server-cont
    networks:
      - miarete

  client:
    build: ./client
    container_name: client-cont
    networks:
      - miarete
    depends_on:
      - server    # il client parte dopo il server
    command: ["server-cont"]
networks:
  miarete:
```

# Docker

---

## ■ Docker-compose

- Permette di descrivere più container in un unico file (docker-compose.yml)
- Definire la reti, immagini, volumi e comandi
- Avviare tutto insieme

## ■ Per ogni service, Compose fa:

- build dell'immagine (se serve)
- docker create del container, specificando --network es-client-server-df\_miarete
- assegna un hostname interno (il nome del service: server, client)
- assegna eventuale container\_name (server-cont, client-cont)
- avvia il container (docker start)

## ■ al momento della creazione del container, viene già “pluggato” nella rete

## ■ quando il processo nel container parte è già connesso alla rete con:

- un IP interno (tipo 172.18.0.x)
- un record DNS interno con il nome del service (server, client)
- se usi container\_name, anche quello è risolvibile (server-cont), a seconda della config

# Docker

---

## ■ Docker-compose

- Permette di descrivere più container in un unico file (docker-compose.yml)
- Definire la reti, immagini, volumi e comandi
- Avviare tutto insieme

## ■ Per ogni service, Compose fa:

- build dell'immagine (se serve)
- docker create del container, specificando --network es-client-server-df\_miarete
- assegna un hostname interno (il nome del service: server, client)
- assegna eventuale container\_name (server-cont, client-cont)
- avvia il container (docker start)

## ■ depends\_on

- Avvia prima il container del server prima di avviare il client

# Docker

---

## ■ Esercizio

- Scaricare il file server2.c che rimane in attesa di connessioni
- Aggiungere una directory server2/ e mettere il relativo Dockerfile
- Modificare docker-compose per inserire il nuovo server (senza modificare il vecchio server)
- Ricompilare il server2
  - docker compose build server
- Lanciare più client
  - docker compose up --scale client=5
  - lancia 5 client (ma bisogna non dare il nome al container)
- Per tornare alla configurazione precedente
  - docker compose down
  - docker compose build server (con la versione precedente)

# Docker

---

## ■ Limiti sulle risorse

- Utilizzo dei cgroups in docker
- Limitare l'utilizzo di RAM del container
  - `docker run --rm -it --memory=50m ubuntu bash`
  - Il kernel lo uccide se supera 50 MB → **OOM killer del cgroup**
- Limitare la CPU
  - `docker run -d -name cpu-test-c --rm -it --cpus=0.05 ubuntu bash`
  - massimo 5% di 1 core
  - `docker stats`, `docker ps`, `docker stop cpu-test-c`
- Limitare i processi
  - `docker run --rm -it --pids-limit=10 ubuntu bash`
  - massimo 10 processi nel container
    - # Esegui questo DENTRO il container
    - `for i in {1..15}; do sleep 30 & done`

# Docker

---

## ■ Limiti sulle risorse

- Utilizzo dei cgroups in docker
- Limitare l'utilizzo di RAM del container
  - `docker run --rm -it --name test-ram-con --memory=50m --memory-swap=50m ubuntu bash`
  - Il kernel lo uccide se supera 50 MB → **OOM killer del cgroup**
- Demo:
  - `cat /sys/fs/cgroup/memory.max`
  - Da altra shell: `docker cp test-ram test-ram-con:/test-ram`
  - Da prima shell: lancio del programma c

# Docker

---

## ■ Limiti sulle risorse

- Utilizzo dei cgroups in docker
- Limitare l'utilizzo di PID del container
  - `docker run --rm -it --name test-pid-con --pids-limit=10 ubuntu bash`
  - Il kernel lo uccide se supera 10 processi
- Demo:
  - Da altra shell: `docker cp test-pid test-pid-con:/test-pid`
  - Da prima shell: lancio del programma c

# Docker

## ■ Limiti sulle risorse con docker-compose

- L'impostazione dei limiti avviene all'interno della definizione del **servizio** nel file `docker-compose.yml`

| Risorsa        | Direttiva                  | Descrizione & Esempio                                                                              | Note per la Lezione                                                                                                              |
|----------------|----------------------------|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Memoria (RAM)  | <code>mem_limit</code>     | Limite rigido della RAM. Oltrepassato, attiva l'OOM Killer.                                        | Usa suffissi come m (MB) o g (GB).                                                                                               |
| Swap (Scambio) | <code>memswap_limit</code> | Limite totale (RAM + Swap). Se uguale a <code>mem_limit</code> , disabilita lo swap nel container. | <b>Cruciale:</b> Per avere un limite RAM <i>rigido</i> (es. 100MB), imposta <code>memswap_limit</code> a <code>\$100m\$</code> . |
| Processi (PID) | <code>pids_limit</code>    | Limita il numero totale di processi e thread attivi nel container.                                 | Ideale per prevenire fork bomb o limitare server multiprocesso (come nel nostro <code>server3.c</code> ).                        |
| CPU (Frazione) | <code>cpus</code>          | Frazione del tempo di un core della CPU che il container può usare.                                | <b>1.0</b> = 1 core intero; <b>0.5</b> = 50% di un core; <b>0.05</b> = 5% di un core.                                            |
| CPU (Priorità) | <code>cpu_shares</code>    | Priorità di scheduling (peso) della CPU. Default: 1024.                                            | Se il tuo servizio ha 2048 e un altro ne ha 1024, riceverà il doppio del tempo CPU disponibile.                                  |

# Docker

---

## ■ Limiti sulle risorse con docker-compose

- L'impostazione dei limiti avviene all'interno della definizione del **servizio** nel file docker-compose.yml

```
services:
  server_limitato:
    build: .
    ports:
      - "5000:5000"

    # LIMITI DI MEMORIA (100 MB RAM MASSIMI)
    mem_limit: 100m
    memswap_limit: 100m    # Impone un limite RIGIDO di 100MB (RAM + Swap)

    # LIMITI CPU
    cpus: 0.2              # Limita l'utilizzo della CPU al 20% di un core

    # LIMITI DI PROCESSO
    pids_limit: 15         # Massimo 15 processi simultanei

    networks:
      - miarete

networks:
  miarete:
    driver: bridge
```

# Docker

---

## ■ Esercizio

- Scaricare il file server-m.c multiprocesso
- Creare la struttura client/server e mettere i relativi Dockerfile
- Modificare docker-compose per gestire il nuovo server imponendo
  - pids\_limits:6, mem\_limit:100m, memswap\_limit:100m
- Ricompilare tutto
  - docker compose build
- Lanciare più client
  - docker compose up --scale client=10
  - lancia 10 client (ma bisogna non dare il nome al container)
- Per tornare alla configurazione precedente
  - docker compose down
  - docker compose build server (con la versione precedente)

# Docker

---

## ■ CPU limits

- Il kernel Linux assegna tempo CPU secondo una quota configurata.
- Due tipi di limiti
  - **CPU share / quota** (quanto tempo CPU puoi usare)
    - `--cpus=N`
    - `--cpu-quota` + `--cpu-period`
    - Il container può usare al massimo N core-equivalenti *di tempo CPU*, distribuiti su tutti i core disponibili.
    - `cpu-period` = durata della finestra di controllo in cui il kernel misura il tempo CPU, default è `--cpu-period = 100000 µs` (100 ms)
    - `cpu-period` ∈ [1000 µs ... 1,000,000 µs], quindi min 1 ms, max 1 sec
  - **CPU affinity** (su quali core puoi girare)
    - `--cpuset-cpus="1,3"`
    - Vincola il container a specifici core logici della macchina.

# Docker

---

## ■ CPU limits

- Il kernel Linux assegna tempo CPU secondo una quota configurata.
  - **CPU share / quota** (quanto tempo CPU puoi usare)
    - `docker run --cpus=1 myapp`
    - `docker run --cpu-period=100000 --cpu-quota=100000 myapp`
      - » periodo e quota in microsecondi
      - » finestra temporale da 100 ms, tempo di CPU 100 ms
    - Se 8 core il container usa al massimo 1/8 della capacità totale
    - `docker run --cpus=0.5 myapp`
    - `docker run --cpu-period=100000 --cpu-quota=50000 myapp`
    - `docker run --cpu-period=200000 --cpu-quota=100000 myapp`
    - massimo 50% di un core logico
    - `docker run --cpus=2 myapp`
    - `docker run --cpu-period=100000 --cpu-quota=200000 myapp`
    - Massimo 2 core logici

# Docker

---

## ■ CPU limits

- Il kernel Linux assegna tempo CPU secondo una quota configurata.
  - **CPU affinity** (su quali core puoi girare)
    - `docker run --cpuset-cpus="0" myapp`
      - » Solo sul core 0
    - `docker run --cpuset-cpus="0,2" myapp`
      - » Solo su core 0 e 2
    - `docker run --cpuset-cpus="0-3" myapp`
      - » Su core 0,1,2,3
    - Uso combinato di affinity e quota
    - `docker run --cpuset-cpus="2,3" --cpus=0.5 myapp`
  - Sono core logici (con hyperthreading vede i core logici separati per ogni hw thread)
  - Il kernel rispetta i vincoli finché possibile, ma può derogare

# Docker

---

## ■ Memory limits

- Docker usa i **cgroups memory** per limitare:
  - memoria fisica (RAM)
  - page cache
  - memory + swap
- Il limite è **hard**: se il container supera la memoria assegnata → OOM.
- Esempio
  - `docker run --memory=512m myapp`
  - Il container può usare al massimo 512 MB di RAM (fisica + page cache).
  - `docker run --memory=512m --memory-reservation=256m myapp`
    - Il kernel prova a tenere *sotto* la reservation, ma può superarla finché c'è RAM
  - `docker run --memory=256m --memory-swap=512m myapp`
    - Può usare 256 MB RAM + 256 MB swap.
  - `--oom-kill-disable` (sconsigliato)

# Docker

---

## ■ Memory limits

- Docker usa i **cgroups memory** per limitare:
  - memoria fisica (RAM)
  - page cache
  - memory + swap
- Il limite è **hard**: se il container supera la memoria assegnata → OOM.
- Memoria fisica limitata:
  - pagine anonime residenti (RAM fisica per heap, stack, allocazioni reali)
  - page cache attribuita al container
  - memory-mapped pages
  - shared memory (shm)
  - tmpfs usato nel container

# Docker

---

## ■ Volumi

- Docker è progettato per essere **effimero** (monouso). Quando un container viene eliminato, tutti i dati scritti al suo interno (sul *layer di scrittura*) vengono persi.
  - Problema: Come possiamo salvare un database, i log o i file di configurazione?
  - Soluzione: I Volumi. Sono il meccanismo preferito di Docker per far sì che i dati sopravvivano al ciclo di vita del container.

| Tipo di Storage                               | Definizione                                                                                 | Scenario Ideale                                                     | Sintassi in Compose                  |
|-----------------------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------|
| <b>1. Named Volumes<br/>(Volumi Nominati)</b> | Aree gestite interamente da Docker sul filesystem dell'host (Docker gestisce la posizione). | <b>Produzione:</b> Database, dati utente, log critici.              | volumes: -<br>db_data:/var/lib/mysql |
| <b>2. Bind Mounts<br/>(Mount di Bind)</b>     | Collegano una <b>directory specifica sull'Host</b> a una directory specifica nel Container. | <b>Sviluppo:</b><br>Sincronizzazione immediata del codice sorgente. | volumes: -<br>./src:/app/code        |

# Docker

---

## ■ Persistenza

- **Bind Mounts:** collega una cartella o file dell'host dentro un container
  - va specificato il path dell'host quindi è host-specific
- Due modalità:
  - `docker run -v <host-path>:<container-path> image`
  - `docker run --mount type=bind,source=<host_path>,target=<container_path> image`
- Esempio
  - `mkdir demo`
  - `echo "test" > demo/file1.txt`
  - `docker run -it --rm --mount type=bind,source=$(pwd)/demo,target=/data alpine sh`
  - `ls /data`
  - Su altro terminale `echo "prova2" > demo/file2.txt`

# Docker

---

## ■ Volumi

- Volume Bind Mounts per i log

services:

server:

build: ./server

container\_name: server-cont

ports:

- "5000:5000"

# === IL BIND MOUNT PER I LOG ===

volumes:

# L'host locale crea la cartella 'log' e la collega a '/app/log' nel container.

# Ogni cosa scritta in '/app/log/server.log' apparirà in './log/server.log' sull'host.

- ./log:/app/log

pids\_limit: 6

networks:

- miarete

restart: always

# ... (Definizione del servizio client e networks) ...

- **./log (Host):** nuova cartella che verrà creata automaticamente nella directory dove si lancia Docker Compose
- **/app/log (Container):** directory nel container in cui il server (o qualsiasi altra applicazione) scrive i file di log

# Docker

---

## ■ Volumi

- Named Volume
- Oggetto docker (gestito dentro docker) **indipendente dal filesystem dell'host**
- Condivisi in modalità più sicura tra container
- E' oggetto docker quindi va creato
  - `docker create volume <vol_name>`
  - `docker run --mount type=volume,src=<vol_name>,dst=<container_path>,...`
  - non c'è path dell'host quindi è portabile su qualunque host

# Docker

---

## ■ Volumi

- Named volumes:
  - `docker volume --help`
    - `create`            crea un volume
    - `inspect`          analizza info sui volumi
    - `ls`                lista volumi
    - `prune`            cancella i volumi non utilizzati
    - `rm`                rimuove un volume
  - Esempio:
    - `docker volume create mio_volume`
    - `docker volume ls`
    - `docker volume inspect mio_volume`
    - `docker volume rm mio_volume`

# Docker

---

## ■ Volumi

- Named volumes:
  - `docker volume --help`
    - `create`            crea un volume
    - `inspect`          analizza info sui volumi
    - `ls`                lista volumi
    - `prune`            cancella i volumi non utilizzati
    - `rm`                rimuove un volume
  - Esempio:
    - `docker volume create my_vol`
    - `docker run -it --name a1 --mount type=volume,source=my_vol,target=/data alpine sh`
    - `echo "PROVA VOLUME" > /data/file.txt`
    - `docker rm -f a1`
    - `docker run -it --a2 --mount type=volume,source=my_vol,target=/data alpine sh`
    - `ls /data`
    - `cat /data/file.txt`

# Docker

---

## ■ Volumi

- Named volumes:
  - `docker volume --help`
    - `create`            crea un volume
    - `inspect`          analizza info sui volumi
    - `ls`                lista volumi
    - `prune`            cancella i volumi non utilizzati
    - `rm`                rimuove un volume
  - Esempio:
    - Fuori dal container si può distruggere il volume
    - Purchè non sia usato da altri container
    - Con `docker volume rm <nome_volume>` si cancella tutto anche con file interni
    - Con `docker volume prune` si cancella qualunque volume non usato

# Docker

---

## ■ Volumi

- Esempio 2:
  - `docker volume create logdata`
  - `docker run -it --name writer1 --mount type=volume,source=logdata,target=/data alpine sh`

Dentro il container

- `echo "Log dal container $(hostname)" >> /data/app.log`
- `echo "Prima riga di log" >> /data/app.log`
- `cat /data/app.log`

Secondo container

```
docker run -it --name writer2 --mount  
type=volume,source=logdata,target=/data alpine sh
```

```
echo "Linea da writer2" >> /data/app.log
```

# Docker

---

## ■ Volumi

### • Esempio 3:

- `docker volume create sqlitedata`
- `docker run -it --name db1 --mount type=volume,source=sqlitedata,target=/data alpine sh`

### Dentro il container

- `apk add --no-cache sqlite`
- `sqlite3 /data/demo.db "CREATE TABLE utenti (id INTEGER PRIMARY KEY, nome TEXT);"`
- `sqlite3 /data/demo.db "INSERT INTO utenti (nome) VALUES ('Alice'), ('Bob');"`
- `sqlite3 /data/demo.db "SELECT * FROM utenti;"`
- `ls /data`
- `exit`

### Secondo container

- `docker run -it --name db2 --mount type=volume,source=sqlitedata,target=/data alpine`
- `apk add --no-cache sqlite`
- `sqlite3 /data/demo.db "SELECT * FROM utenti;"`

# Docker

---

## ■ Volumi

- Named Volume è un metodo di archiviazione gestita interamente da Docker.

services:

server:

build: ./server

container\_name: server-cont

# 1. COLLEGARE IL VOLUME AL SERVIZIO

volumes:

# Collega il volume 'app\_data' alla cartella di configurazione interna al container.

# Ogni configurazione salvata in '/app/config' è persistente.

- app\_data:/app/config

pids\_limit: 6

networks:

- miarete

# 2. DICHIARAZIONE GLOBALE DEL VOLUME

volumes:

# Il volume nominato 'app\_data' viene creato e gestito da Docker.

app\_data:

# Opzionale: aggiungere qui driver o opzioni specifiche, ma per semplicità basta il nome

# Docker

---

## ■ Volumi

services:

# SERVIZIO A: Crea il file

creatore:

image: alpine

container\_name: volume\_creatore

volumes:

# Monta il volume 'dati\_persistenti' nella cartella /data del container

- dati\_persistenti:/data

# Comando: Crea un file chiamato 'test.txt' e scrive l'ora attuale al suo interno

command: ["sh", "-c", "echo 'Creato il file' > /data/test.txt && echo 'File creato! esco'"]

# SERVIZIO B: Legge il file (da lanciare separatamente)

lettore:

image: alpine

container\_name: volume\_lettore

volumes:

# Monta lo stesso volume 'dati\_persistenti' nella cartella /data

- dati\_persistenti:/data

# Comando: Legge il contenuto del file 'test.txt'

command: cat /data/test.txt

# DICHIARAZIONE DEL VOLUME NOMINATO

volumes:

dati\_persistenti:

driver: local # Opzione predefinita per l'archiviazione locale

# Docker

---

## ■ Volumi

- Prova volume

# 1. Avvia il container 'creatore'.

# Il volume 'dati\_perso' viene creato se non esiste.

`docker compose up creatore`

# 2. Rimuovi il container 'creatore'. Questo simula la perdita del container.

# ATTENZIONE: Il volume DATI\_PERSO NON viene rimosso!

`docker rm volume_creatore`

# Avvia il container 'lettore' e cerca di leggere il file creato nel Passo A.

`docker compose up lettore`

`docker compose down -v`

# Docker

---

- **Volumi**

- Verificare il mount point

`docker volume ls`

`docker volume inspect <nome del volume>`

Cercare "mount point"

# Docker

---

## ■ Volumi

- Driver dei volumi

Docker permette anche driver diversi (NFS, CIFS, cloud storage...), ma local è la scelta standard:

```
volumes:  
  dati_persistenti:  
    driver: local
```

In NFS:

```
volumes:  
  volume_nfs:  
    driver: local  
    driver_opts:  
      type: nfs  
      o: addr=192.168.1.20,rw  
      device: ":/exports/data"
```